○ E-LEARNING · STUDY NOTES

Nicloud · Kubernetes on AWS

Module 3 – Service mesh, network policy and secrets

Contents

Learning objectives 2
Architecture 2
Service mesh 2
Istio 3
Linkerd 3
Istio or Linkerd 4
Calico: network policies 5
Example: deny all, allow what is needed 5
Vault in high availability 5
Secrets Store CSI Driver 7
Vault or Secrets Manager 7
Summary 8
Quiz 8
Additional resources 9

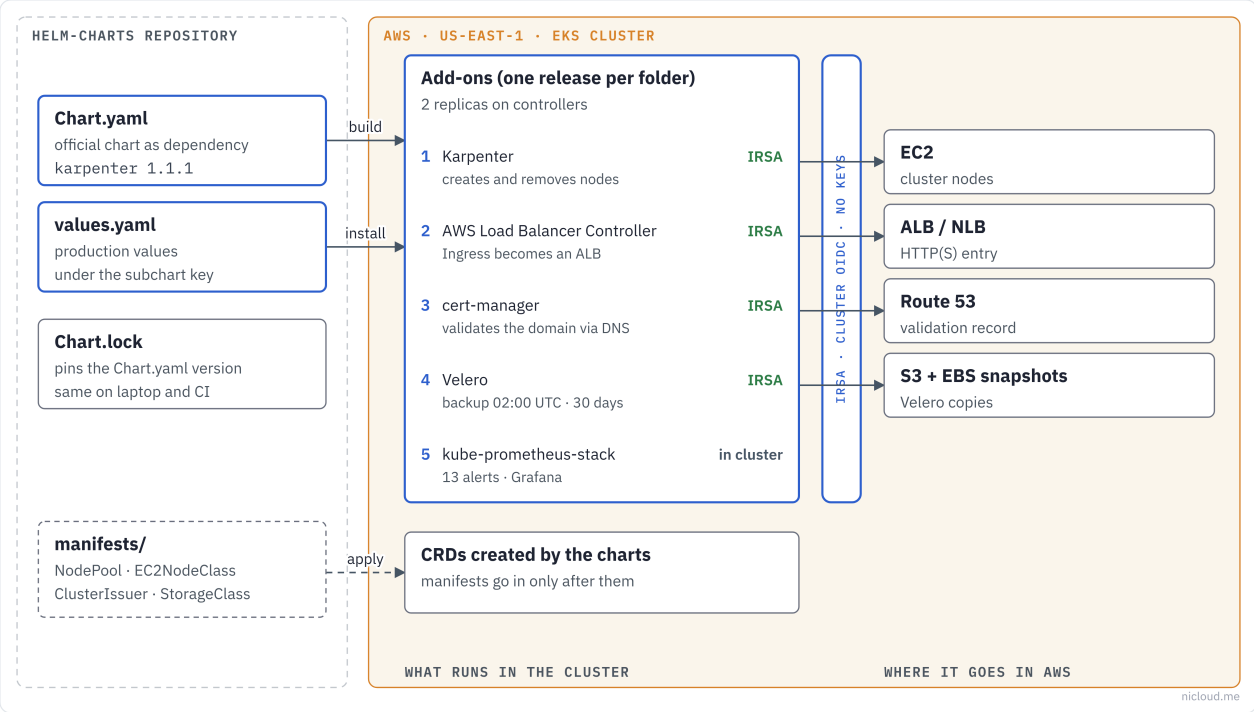
Learning objectives

In this module, you will learn to:

- Explain what a **service mesh** adds: **mTLS**, per-service metrics and traffic control.
- Install **Istio** and **Linkerd** in the right order and say when to choose each.
- Describe Linkerd's certificate chain (**trust anchor** and **issuer**) and why it must not live in the repository.
- Apply **NetworkPolicies** with **Calico** on top of the VPC CNI and build the "deny all, allow what is needed" model.
- Run **Vault** in high availability with **Raft** storage: initialize, unseal and join nodes.
- Mount **Secrets Manager** secrets as files with the **Secrets Store CSI Driver**.
- Choose between Vault and Secrets Manager for each kind of secret.

Architecture

The diagram shows the overview. This module covers the layer **between** pods: who may talk to whom (network and mesh) and with which credentials (secrets).



Service mesh

A **service mesh** puts a **proxy** next to every pod, the **sidecar**. All traffic between services goes through the proxies, which gives you three things without changing a line of application code:

- **mTLS** (mutual TLS): each pod gets an identity with a certificate, and the connection between two pods is encrypted and authenticated on both sides.

- **Golden metrics per service**: success rate, latency and volume between every pair of services.
- **Traffic control**: retries, timeouts, weighted splitting for canaries.

The cost is one more proxy per pod (memory and a few milliseconds) and one more component to operate. If all you need is to block traffic between namespaces, NetworkPolicy does it without a mesh.

Istio

Istio uses **Envoy** as its proxy and has the most complete feature set: fault injection, traffic mirroring, L7 authorization, ingress and egress gateways.

The repository splits Istio into two charts, which must be installed in this order:

Order	Folder	What it installs
1	`istio/istio-base`	CRDs and cluster-wide resources
2	`istio/istiod`	the control plane, which configures the proxies and issues certificates

```
# istio/istiod/values.yaml
istiod:
  pilot:
    resources:
      requests:
        cpu: 200m
        memory: 256Mi
  meshConfig:
    accessLogFile: /dev/stdout
```

- `pilot.resources` reserves CPU and memory for istiod. It recomputes the configuration of every proxy on each change, and without a reservation it competes with the applications.
- `accessLogFile: /dev/stdout` turns on the proxies' access log. Each request becomes a line in the sidecar log, which the Fluent Bit in module 4 ships to CloudWatch. It helps a lot with troubleshooting, but it raises log volume.

To add a namespace to the mesh: `kubectl label namespace <ns> istio-injection=enabled` and restart the pods. The sidecar is only injected when a pod is created.

Linkerd

Linkerd uses its own proxy, written in Rust, smaller and simpler than Envoy. It turns on mTLS by itself for every meshed pod and delivers the golden metrics without configuration.

It also has two charts, in this order:

Order	Folder	What it installs
1	`linkerd/linkerd-crds`	CRDs
2	`linkerd/linkerd-control-plane`	control plane and identity

Certificate chain

Linkerd's mTLS depends on two certificates that **you** generate before installing:

- The **trust anchor**: the root of trust of the mesh, long-lived (for example, 10 years). It goes into `identityTrustAnchorsPEM`.
- The **issuer**: an intermediate certificate, signed by the trust anchor and shorter-lived (for example, 1 year). The control plane uses the issuer to sign each pod's certificate, which lasts only 24 hours.

```
# linkerd/linkerd-control-plane/values.yaml (excerpt)
linkerd-control-plane:
  identityTrustAnchorsPEM: |
    -----BEGIN CERTIFICATE-----
    CHANGE_ME
    -----END CERTIFICATE-----
```

The issuer's private key does **not** go into the versioned `values.yaml`. Pass it at install time (`--set-file identity.issuer.tls.keyPEM=issuer.key`) or, better, let cert-manager generate and rotate the issuer.

The CI test needs to render the manifests, so it uses `ci/test-values.yaml` with fake certificates. They make `helm template` pass and are good for nothing else.

Linkerd version

The Linkerd project publishes open source releases only on the **edge** channel, with date-based versions (`2026.11.1`). The repository used `version: "*"`, which pulled the newest version on every `helm dependency update`. The version is now pinned, and upgrading is a decision with a commit (module 1).

Istio or Linkerd

	Istio	**Linkerd**
Proxy	Envoy (C++)	linkerd2-proxy (Rust)
Memory per sidecar	higher	lower
mTLS	automatic	automatic
Fault injection, mirroring	yes	no
Learning curve	long	short
Choose when	you need fine-grained traffic control or already use Envoy	you want mTLS and metrics with minimal operations

Do not install both in the same cluster. Both charts are here for study and to choose per cluster.

Calico: network policies

By default in Kubernetes, **every pod can talk to every pod**. A **NetworkPolicy** restricts that, but Kubernetes only stores the rule. The network plugin is what enforces it.

On EKS, pod networking comes from the AWS **VPC CNI** (each pod has a VPC IP). **Calico** comes in only as the **policy engine**, without replacing the network:

```
# calico/values.yaml
tigera-operator:
  installation:
    kubernetesProvider: EKS
    cni:
      type: AmazonVPC
```

- `cni.type: AmazonVPC` tells Calico to keep the VPC CNI and only enforce policies. Replacing the EKS CNI is a different, much riskier conversation.
- The chart installs the **tigera-operator**, which in turn installs and upgrades Calico.

EKS can also enforce NetworkPolicy in the VPC CNI itself. Calico is still useful when you need its own policies (`GlobalNetworkPolicy`, DNS-based rules), which go beyond the standard API.

Example: deny all, allow what is needed

```
# calico/manifests/default-deny.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: default
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

- `podSelector: {}` selects **every** pod in the namespace.
- `policyTypes: [Ingress]` with no `ingress` rules means: no inbound traffic is allowed.

From there, each service gets a policy that allows only those who need to reach it. Notice this policy closes only **inbound** traffic. Outbound stays open. To close outbound too, add `Egress` and remember to allow DNS (port 53 to CoreDNS), or no pod can resolve names.

Vault in high availability

HashiCorp **Vault** stores secrets and also **generates them on demand**: a database password valid for one hour, a certificate from an internal PKI, temporary AWS credentials. That is what sets it apart from a store of fixed values.

```
# vault/values.yaml (excerpt)
vault:
  server:
    ha:
      enabled: true
      replicas: 3
      raft:
        enabled: true
        setNodeId: true
    dataStorage:
      size: 10Gi
      storageClass: gp3
    auditStorage:
      enabled: true
      size: 10Gi
  injector:
    enabled: true
  ui:
    enabled: true
    serviceType: ClusterIP
```

- `ha.replicas: 3` with **Raft**: the three pods replicate data among themselves, with no separate Consul. One is the leader and the others follow. Three is the minimum for the cluster to survive one node down, because Raft needs a majority (2 of 3).
- `dataStorage` and `auditStorage` create one **gp3** volume per pod for data and another for the **audit log**, which records every secret access.
- `injector.enabled` turns on the **Agent Injector**: with annotations on the pod, a sidecar fetches the secret from Vault and writes it to a file for the application to read.
- `ui.serviceType: ClusterIP` keeps the UI off the Internet. Reach it with `kubectl port-forward`.

Until November 2026, this `values.yaml` turned on HA without turning on Raft, even though the README said "HA with Raft". `helm template` passed, but the pods would have had nowhere to store data together. It is the kind of mistake lint and template do not catch: the chart is valid, the configuration is incomplete.

How it works

(1) Initialize

`vault operator init` on the first pod generates the **unseal keys** and the root token. Keep the keys apart, with different people.

(2) Unseal

Every Vault pod starts **sealed**: the data is encrypted and it does not answer. `vault operator unseal`, with the minimum number of keys, opens it. This repeats on every pod restart.

(3) Join the other nodes

`vault operator raft join http://vault-0.vault-internal:8200` on pods 1 and 2, then unseal each one. `vault operator raft list-peers` shows all three.

Unsealing by hand on every restart does not scale. In production, use **auto-unseal** with AWS KMS: Vault asks KMS to unseal it, through an IRSA role, and restarts on its own.

Secrets Store CSI Driver

The **Secrets Store CSI Driver** mounts secrets from an external store as **files** inside the pod. On AWS, together with the AWS provider, it reads **Secrets Manager** and **Parameter Store**.

```
# secrets-csi/values.yaml
secrets-store-csi-driver:
  syncSecret:
    enabled: true
  enableSecretRotation: true
```

- `syncSecret.enabled` lets you also copy the secret into a Kubernetes Secret, for workloads that need it as an environment variable.
- `enableSecretRotation` re-reads the store periodically and updates the mounted file when the secret changes.

What to mount is described in a **SecretProviderClass**:

```
# secrets-csi/manifests/example-spc.yaml (excerpt)
kind: SecretProviderClass
spec:
  provider: aws
  parameters:
    objects: |
      - objectName: "arn:aws:secretsmanager:us-east-1:123456789012:secret:my-
secret"
        objectType: "secretsmanager"
```

The chart installs only the driver. The **AWS provider** (`secrets-store-csi-driver-provider-aws`) is installed separately, as the folder's README shows. The **application pod** is what reads the secret, so its service account is the one that needs IRSA with `secretsmanager:GetSecretValue` on that secret.

With `syncSecret`, the value exists again in a Kubernetes Secret, stored in etcd and readable by anyone with access to the namespace. Use it only when the application cannot read a file.

Vault or Secrets Manager

Situation	Best choice
Fixed password, third-party API key	Secrets Manager + CSI: managed by AWS, nothing to operate
Automatic RDS password rotation	Secrets Manager (native rotation)
Database credentials valid for one hour, per pod	Vault (dynamic secrets)
Internal PKI, short-lived certificates	Vault, or cert-manager with its own issuer
Several clouds or on-premises	Vault

Vault does more and charges in operations: three pods, unsealing, Raft backups, upgrades. If the need fits Secrets Manager, that is the cheaper path in team time.

Summary

A **service mesh** puts a proxy next to every pod and delivers **mTLS**, per-service metrics and traffic control without changing the application. **Istio** has more features and more parts. **Linkerd** is lighter and turns on mTLS by itself, but requires you to generate the **trust anchor** and the **issuer**, whose key never goes into the repository. Both are installed in two steps, CRDs first.

Calico, on top of the VPC CNI, enforces **NetworkPolicies**. The safe model starts with a policy that denies all inbound traffic in the namespace and then allows service by service. When closing outbound traffic, allow DNS.

For secrets, **Vault** in HA with **Raft** and three replicas generates credentials on demand and records every access, but it must be initialized, unsealed (ideally with KMS) and maintained. The **Secrets Store CSI Driver** mounts **Secrets Manager** secrets as files, with the permission on the application's own service account. For most fixed secrets, Secrets Manager is enough.

Quiz

You installed `istiod` and it failed with unknown types. What was missing?

- Installing `istio-base` first. It creates the CRDs istiod uses.

You labeled the namespace with `istio-injection=enabled`, but the pods still have no sidecar. Why?

- The sidecar is only injected when a pod is created. Restart the namespace's deployments.

Where should the Linkerd issuer's private key live?

- Outside the repository: passed at install time with `--set-file`, or generated and rotated by cert-manager. `ci/test-values.yaml` only holds fake certificates for CI.

A team wants mTLS between all services and per-service metrics, with minimal operations. Istio or Linkerd?

- Linkerd. It turns on mTLS by itself, delivers the metrics without configuration and has fewer parts. Istio only pays off if they need features Linkerd lacks, such as fault injection.

You applied the `default-deny-ingress` policy in the `default` namespace. Pod A can still reach the Internet. Is that right?

- Yes. The policy closes only inbound traffic (`policyTypes: [Ingress]`). To close outbound, add `Egress` and allow DNS.

You added `Egress` to a deny-all policy and every pod started failing to call other services by name. What did you forget to allow?

- DNS: outbound to CoreDNS on port 53 (UDP and TCP).

The Vault cluster has three Raft replicas. How many pods can fail without Vault stopping?

- One. Raft needs a majority (2 of 3) to elect a leader and write.

After a maintenance window, all three Vault pods came back, but no application can read secrets. What probably happened?

- The pods came back sealed. Without auto-unseal, you must run `vault operator unseal` on each one. With KMS auto-unseal, this does not happen.

An application uses the repository's SecretProviderClass and fails with AccessDenied when mounting the volume. Which service account needs the permission?

- The application's own, with IRSA and `secretsmanager:GetSecretValue` on that secret. The driver reads the secret on behalf of the pod.

Additional resources

- Istio: install with Helm
- Linkerd: install with Helm
- Linkerd: generating certificates
- Calico on EKS
- Network Policies (Kubernetes)
- Vault on Kubernetes with Raft
- Vault: auto-unseal with AWS KMS
- Secrets Manager with the CSI Driver on EKS